

NLP 2025 Lab 3 Report: Group 35

RSAA (rowaid) Asaba

KM (Kordian) Klimas

Tikhon Magafurov

1 Exercise 1: Unpacking the set

We unpack the 'set' (containing tuples of full and compressed sentences) column into separate 'sentence' and 'compressed' columns for ease of use.

```
def unpack(examples):
    sentences = []
    comp_sentences = []

    for set in examples['set']:
        sentences.append(set[0])
        comp_sentences.append(set[1])

    examples['sentence'] = sentences
    examples['compressed'] = comp_sentences
    return examples
```

2 Exercise 2: Questions about the tokenizer

The vocabulary size is 30522, including all the special tokens.

Apart from [CLS] and [SEP], there are 3 other special tokens: [PAD], [UNK] and [MASK]:

- [UNK] replaces all the rare, OOV words in the dataset. So if a word in the input is not recognized, it is tokenized as [UNK]
- [PAD] is padding to keep sentence length constant during training
- [MASK] is used during the pretraining of the BERT model. Tokens in the training data are randomly replaced with [MASK] and the model is optimized to be able to better predict/reconstruct the original token based on its context.

3 Exercise 3: Questions about the Model

3.1 What is the number of transformer layers in this model?

There are 12 transformer layers in this model

3.2 What is the dimension of the embeddings?

The dimension of the embeddings is 768

3.3 What is the hidden size of the FFN in the transformer layer?

The hidden size of the FFN in the transformer layer is 3072

3.4 What is the total number of parameters of the model?

The total number of parameters of the model is 109,482,240

3.5 How can you find the vocabulary size from the model?

You can find the vocabulary size of the model by looking at the shape of the word_embeddings tensor. In this case it is (30522, 768) so the vocabulary size is 30522 and the embedding dimension is 768.

4 Exercise 4: Plotting the layer-wise similarities between words

We finished the function that compares the activations of tokens of interest in specified layers

We then applied it to the sentences:

- "We will rob a bank next week!"
- "The children skipped stones by the bank of the river."
- "I put money in the bank."

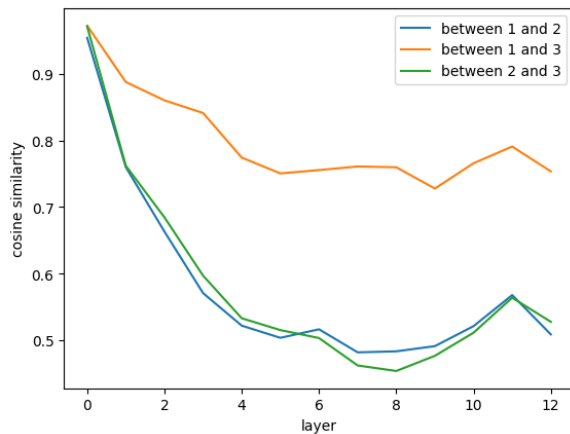


Figure 1: Layer activations corresponding to the "bank" token

5 Exercise 5: Evolution of embeddings

5.1 Discuss the plots above. Are the plots showing what you expected to see?

The plots are indeed show what we'd expect to see. Across all sentences, activations for "bank" between sentence 1 and 3 are most similar because the word "bank" is used in the same way in both sentences. That is, meaning "financial establishment". In sentence 2, the word "bank" is used in a different sense as "land alongside a river or lake". Thus, the activations of "bank" in sentence 2 have lower similarity to that in sentences 1 and 2.

5.2 Take a look at the similarity between the word "bank" in the first example for the first layer. It is close to '1' but not exactly. Why is that?

This is due to the effect of adding positional embeddings to each word embedding to represent the order of the words in the sentence. Since the position index of the word "bank" is different in all 3 sentences (idx=[5,7,6]) the positional embeddings are also different.

5.3 Plot and analyze the similarities between words "nice", "bad", and "lovely". Are the plots showing what you expected to see?

For this exercise we looked at the sentences:

- "The weather is nice today."
- "The weather is bad today."
- "The weather is lovely today."

The results are as expected. The highest similarity is between the synonyms "nice" and "lovely"

across sentences 1 and 3 while the similarity of ("nice", "bad") and ("lovely", "nice") is noticeably lower across all layers)

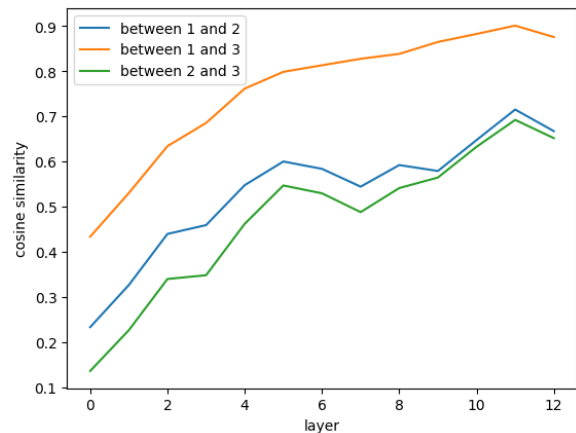


Figure 2: Activation cosine similarity of "nice", "bad", and "lovely" across layers

5.4 Try a different set of sentences and comment on the results

For this exercise we looked at the sentences where the word "light" is used in two different ways. One meaning of the word "light" is as a verb meaning "to ignite" and another is "light" in the sense of "weighing little". The sentences we used are:

- "You have to light the fuse"
- "Did you light the match?"
- "Oh don't worry, the bag is quite light"

As with "bank", the two sentences where "light" is used in the same way maintain the highest similarity.

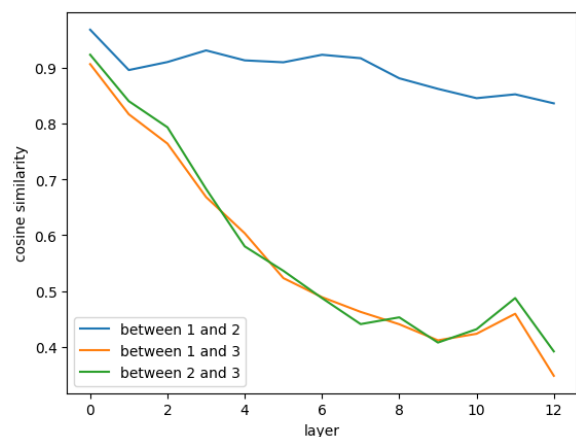


Figure 3: Activation cosine similarity of "light" across layers between the 3 sentences

6 Exercise 6: Implement sentence embeddings

We implemented a function to evaluate the sentence embeddings for a list of sentences, passing the layer to use for that as an argument. The sentence embeddings are thus the mean activations of the specified layer across all tokens. The optional functionality of ignoring [CLS] and [SEP] tokens was also added.

We then evaluated sentence embeddings for the sentences:

- "We will rob a bank next week!"
- "Let's put our savings into a bank account."
- "We will steal some money from the bank."

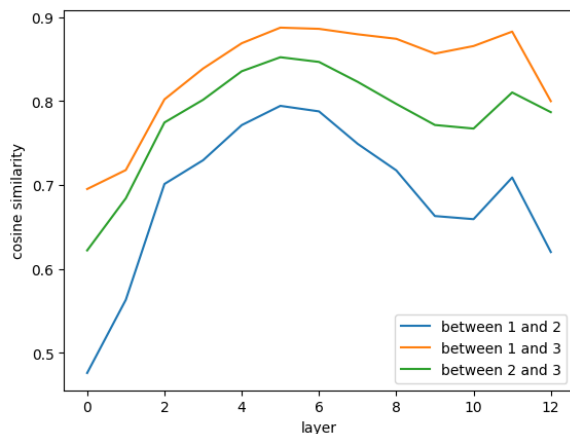


Figure 4: Sentence similarities for sentences from ex. 6

7 Exercise 7: Try different sentences

We the sentence embeddings of a few new sets of sentences to gauge the effectiveness of our method.

First we looked at:

- "I'm dying, man. It's too much!"
- "This is very funny"
- "I was admitted to the ER"

Theses sentences reveal how the network gets better at understanding the meaning and context of a sentence as you step through its layers. To a human, the first sentence may at first seem to be about literal dying but on closer inspection most would see that it is plausibly just someone using hyperbole to express that something is funny. The plot depicts this same dynamic since for the first few layers, sentence 1 is considered most similar

to sentence 3 (related to mortality) but in later layers, as the model processes the sentence further, it becomes most similar to sentence 2 (related to humour).

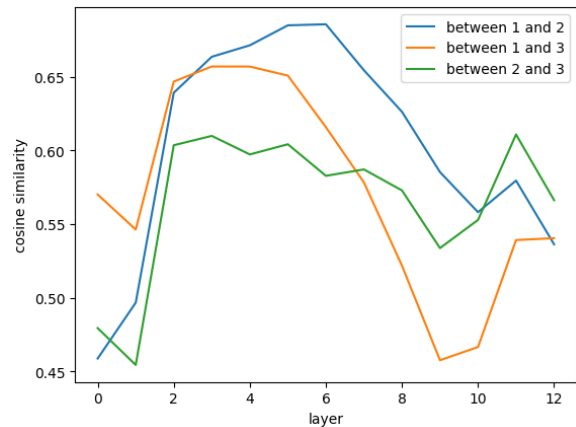


Figure 5: Sentence similarities for sentences from ex. 7 related to humour and mortality

We looked at another 3 sentences (Figure 6):

- "We will have to stay indoors"
- "It is raining"
- "The weather is sunny"

It can be seen that the similarity of sentences 2 and 3 is the highest, probably because they both directly reference weather. This is despite the fact that the two sentences contradict each other. This underscores that this method of sentence embedding does not always match human intuitions about similarity.

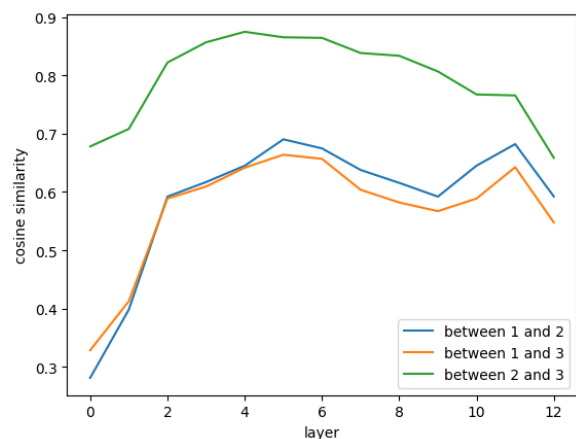


Figure 6: Sentence similarities for sentences from ex. 7 related to the weather

8 Exercise 8: Tokenize sentence and compressed

For this exercise we wrote two functions to tokenize the complete and compressed sentences from the dataset

```
def tokenize_sentence(examples):
    sentences = examples['sentence']
    tokenized_sentence = tokenizer(
        sentences,
        padding=False,
        return_tensors='np'
    )

    return tokenized_sentence

def tokenize_compressed(examples):
    comp_sentence = examples['compressed']
    tokenized_compressed = tokenizer(
        comp_sentence,
        padding=False,
        return_tensors='np'
    )

    return tokenized_compressed
```

We then proceeded to embed them via our sentence embedding method from exercise 6.

```
def tokenize_sentence(examples):
    sentences = examples['sentence']
    tokenized_sentence = tokenizer(
        sentences,
        padding=False,
        return_tensors='np'
    )

    return tokenized_sentence

def tokenize_compressed(examples):
    comp_sentence = examples['compressed']
    tokenized_compressed = tokenizer(
        comp_sentence,
        padding=False,
        return_tensors='np'
    )

    return tokenized_compressed
```

We then proceeded to embed them via our sentence embedding method from exercise 6.

9 Exercise 9 & 10: Embed query, and similarity calculation

We have developed the following components to support our query retrieval system:

1. **Embedding Function** – Converts a query into its vector representation using the model (google-bert/bert-base-uncased).
2. **Cosine Similarity Calculation** – Computes the similarity between the query vector and each of the n document vectors.
3. **Retrieval Function** – Returns the most relevant sentences based on their similarity to the query.

These functions enabled us to evaluate the quality of the retrieved results and conduct further experiments with the embedding-based model. Specifically, we compared its performance and behavior against traditional methods such as *Bag-of-Words* and *TF-IDF*, while also analyzing its unique strengths and limitations.

10 Exercise 11: Analysis of Query Embedding Experiments

10.1 Role of entities, and meaning lying in word ordering

Queries: "DRUNK neighbour attacked a woman" vs. "DRUNK woman attacked a neighbour".

- *Similarity Scores:* The model assigned a higher similarity score to sentences that correctly matched the attacker/victim roles (e.g., $\text{sim} \approx 0.84$ for the matching "DRUNK woman attacked a neighbour..." sentence versus $\text{sim} \approx 0.79$ for the converse).
- *Context Preservation:* Although occasional role flips occurred among the top-ranked results, the embeddings predominantly preserved the overall negative context involving a drunk individual, an attack event, and the entities "woman" and "neighbor with correctly assigned role"
- *Comparison to Traditional Methods:* This behavior indicates a more nuanced understanding of sentence structure and semantic relationships than Bag-of-Words or TF-IDF approaches.

10.2 Sentiment Preservation in Weather Queries

Queries: "the weather is good" vs. "the weather is bad".

- *Sentiment Alignment:* Positive queries retrieved descriptions with favorable weather conditions (e.g., clear skies), whereas negative queries returned adverse phenomena (e.g., drought, snow).
- *Semantic Precision Limits:* Some retrieved sentences were contextually irrelevant (e.g., "What time of the day is best for hunting..."), revealing limitations in the model's precision when handling simple, subjective sentiments.

10.3 Phrase-Specific Retrieval Efficacy

Query: "suddenly I'm dreaming of . . . bacon".

- *High Specificity:* A near-exact phrase match achieved a very high similarity score ($\text{sim} \approx 0.96$).
- *Rapid Relevance Decay:* Relevance drops sharply for non-exact matches, indicating that the embedding strongly privileges this specific semantic unit. In contrast, keyword-based methods (e.g., Bag Of Words) often assign a maximum similarity score of 1 to any document containing all query terms. In certain retrieval scenarios, may be preferable.

11 Exercise 12 & 13. Evaluating Retrieval with Recall@K

In this section, we evaluated the performance of our sentence retrieval system using the test set. Following standard practice in Information Retrieval, we employed Recall@K as the primary evaluation metric. Based on created matrix, we calculated the recall for k most similar sentences.

Retrieval is performed by calculating the cosine similarity between the query embedding and the embeddings of all sentences in the test set. This similarity calculation is efficiently implemented using matrix operations in PyTorch via the `cosine_similarity_m_to_n` function, which computes the similarity between all pairs of vectors from two sets.

Recall@K measures the proportion of relevant documents retrieved within the top K results. The

formula is presented using inline division to better fit within the column width:

$$\text{Recall@K} = \frac{\text{Number of relevant documents retrieved in the top } K}{\text{Total number of relevant documents}}$$

For this evaluation, compressed sentences are queries, and each has exactly one relevant document: the original sentence.

Recall@K Results

The calculated Recall@K scores for different K values are:

- **Recall@1:** 50.66%
- **Recall@5:** 67.65%
- **Recall@10:** 73.71%
- **Recall@20:** 78.97%
- **Recall@50:** 85.00%
- **Recall@100:** 88.84%
- **Recall@1000:** 96.81%
- **Recall@10000:** 99.54%

Commentary

As expected, Recall@K increases with K. This is because a larger K provides a wider search window, making it more likely to include the single relevant sentence. The results show that while the correct sentence isn't always the top result (50.66% at K=1), it is found within the top results with increasing reliability as K grows (up to 99.54% at K=10000).

12 Exercise 14: Different Ways of Embedding Sentences

In this exercise, we experimented with different strategies to compute sentence embeddings from BERT outputs and evaluated their effect on a sentence retrieval task. We implemented the following three approaches:

- **CLS Token Representation (cls):** Directly using the embedding of the [CLS] token from the last hidden layer. This is commonly used as a sentence representation in classification tasks.

- **Mean Pooling of Last Layer (mean_last):** Averaging all token embeddings from the last hidden layer, excluding special tokens such as [CLS], [SEP], and [PAD].
- **Mean Pooling of Last 4 Layers (mean_last4):** Averaging token embeddings across the last four hidden layers before applying mean pooling, providing a richer representation.

Inspection of Representations

Before running over the full dataset, we printed out the shapes and sample values from the first batch to ensure our code was extracting the correct tensors:

```
=== INSPECTION ===
input_ids shape:
  torch.Size([8, 44])
last_hidden_state shape:
  torch.Size([8, 44, 768])
# hidden_states layers: 13
first 3 tokens, dims0-4:
[[ 0.01445213  0.00899917
 -0.3316019  -0.3589843  -0.18942316]
 [-0.05259338 -0.12401631
 -0.44809347 -0.41334617 -0.26303238]
 [-0.10502277  0.1103867
 -0.2477009  -0.18734208 -0.42727262]]
CLS embedding dims0-4:
[ 0.01445213  0.00899917
 -0.3316019  -0.3589843  -0.18942316]
=====
```

This confirms:

- We receive an (8×44) token matrix per batch.
- The model outputs a $(8 \times 44 \times 768)$ tensor of hidden states.
- There are 13 layers of hidden states (embedding + 12 transformer layers).
- The first few dimensions of token vectors and the [CLS] vector contain meaningful values.

We then evaluated these embeddings using Recall@ k on the sentence retrieval task, where the goal is to match compressed sentences with their original counterparts.

Results

Discussion

The results demonstrate that the [CLS] token performs the worst among all strategies, achieving

Embedding Strategy	Recall@1	Recall@5	Recall@10
cls	10.82%	18.24%	21.99%
mean_last	47.06%	63.83%	69.73%
mean_last4	45.07%	61.83%	68.15%

Table 1: Recall@K performance for different sentence embedding strategies

only 10.82% Recall@1. This confirms that while the [CLS] token is effective for classification tasks, it lacks the granularity required for sentence retrieval.

The mean_last strategy yields the best performance across all recall levels. Averaging token embeddings from the last hidden layer captures a more comprehensive view of sentence semantics, and this method proved especially effective on our dataset.

The mean_last4 method also performed well, though slightly below mean_last. This suggests that while combining multiple layers adds information, it might introduce noise or dilute relevant features for this specific task.

Overall, mean pooling—especially from the last layer—offers a simple yet highly effective approach for deriving sentence embeddings from BERT for retrieval tasks.

13 Exercise 15: Using a Retrieval-Fine-Tuned BERT Model

In this exercise, we replaced the vanilla BERT model with the sentence-transformers/msmarco-bert-base-dot-v5 model, which has been fine-tuned on the MS-MARCO passage retrieval task to produce embeddings optimized for ranking via dot-product similarity.

Inspecting Token and Sentence Representations

To verify that we are correctly extracting embeddings:

- We passed a single example sentence through the model and examined its `last_hidden_state` tensor, confirming a shape of

$$(1, L, 768)$$

where L is the tokenized sequence length (e.g. 28 tokens) and 768 is the hidden dimension. - We printed the first five dimensions of the first three token vectors, observing non-trivial positive and negative values, which indicates meaningful contextualization. - We also inspected the first five dimensions of the [CLS] token embedding—this vector is intended to represent the entire sentence after fine-tuning for retrieval.

These checks ensure that the model is producing valid, high-dimensional embeddings both at the token level and at the sentence level.

Retrieval Evaluation

We then applied these normalized [CLS] embeddings to our sentence retrieval task, using compressed sentences as queries and full sentences as the target corpus. On the full 36 000-example test split, we measured Recall@ k :

- **Recall@1:** 89.67%
- **Recall@5:** 95.50%
- **Recall@10:** 96.48%

By comparison, the best vanilla-BERT strategy in Exercise 14 (mean-pooling of the last hidden layer) achieved only about 47% Recall@1. The substantial gains here demonstrate the benefit of task-specific fine-tuning on a large retrieval corpus: the MSMARCO model's embeddings are far more effective at matching semantically equivalent (compressed vs. original) sentences.

Conclusion

Using the MSMARCO-fine-tuned BERT model yields very high retrieval accuracy, thanks to its end-to-end optimization for ranking. Inspecting both token-level and sentence-level outputs confirmed that the model produces meaningful, 768-dimensional vectors. The Recall@ k results validate that a retrieval-trained embedding is superior to general-purpose BERT embeddings for the task of matching compressed queries to their corresponding full sentences.

14 Exercise 16: Comparison Between Models

In this exercise we compare retrieval performance across the three embedding methods:

- **BGE-small-en (Lab 2):** A lightweight Transformer embedding model (BAAI's BGE) that produces 384-dimensional sentence vectors.
- **BERT-Base (mean_last, Exercise 14):** The standard bert-base-uncased model (12 layers, hidden size 768, 12 attention heads), mean-pooling its final hidden states.
- **MSMARCO-BERT (CLS, Exercise 15):** A 12-layer, 768-dimensional BERT model fine-tuned on the MS MARCO retrieval task, using its [CLS] output.

Architectural Summaries

- **BGE-small-en:**
 - Transformer encoder (exact number of layers undisclosed), hidden dimension 384.
 - Optimized for sentence embedding tasks, small footprint.
- **BERT-Base (bert-base-uncased):**
 - 12 Transformer layers, each with 12 attention heads.
 - Hidden size 768, intermediate feed-forward dim 3072.
 - Pre-trained on Masked Language Modeling + Next Sentence Prediction.
- **MSMARCO-BERT (msmarco-bert-base-dot-v5):**
 - Same backbone as BERT-Base (12 layers, 768-dim hidden).

- Fine-tuned on millions of (query, passage) pairs under a dot-product ranking loss.
- Produces [CLS] vectors directly usable for retrieval.

Recall@ k Performance

Model	Recall@1	Recall@5	Recall@10
BGE-small-en (384 d)	98.06%	99.29%	99.44%
BERT-Base (mean_last, 768 d)	47.06%	63.83%	69.73%
MSMARCO-BERT (CLS, 768 d)	89.67%	95.50%	96.48%

Table 2: Recall@ k on the 36 000-example test set for the three embedding methods.

Discussion

BGE-small-en With its compact 384-dimensional embeddings, BGE-small-en delivers near-perfect retrieval ($R@1 \approx 98\%$). Its small size and high accuracy make it ideal for large-scale or latency-sensitive applications.

MSMARCO-BERT Fine-tuned on a massive retrieval corpus, it achieves strong performance ($R@1 \approx 90\%$), significantly outperforming vanilla BERT-Base. The retrieval objective sharpens its ability to align queries with passages, at the cost of increased model size and inference time compared to BGE-small-en.

BERT-Base (mean_last) Out-of-the-box BERT-Base with mean-pooling reaches only $R@1 \approx 47\%$. While its contextual embeddings capture general language structure, the lack of retrieval-specific fine-tuning limits its matching accuracy.

Conclusions

- *BGE-small-en* combines a small footprint with exceptional retrieval accuracy, making it the clear winner for this particular task.
- *MSMARCO-BERT* validates the power of retrieval-oriented fine-tuning on large corpora, achieving high recall at moderate cost.
- *BERT-Base* offers a strong general-purpose encoder but requires further fine-tuning or enhanced pooling strategies to match the performance of specialized models.

A Collaborators outside our group

No external collaborators.

B Use of genAI

We leveraged ChatGPT to help debug code, enhance clarity and accuracy, and refine LaTeX formatting and implementation details throughout this report.